

Care-Sync / M-Health

A Privacy-First Platform for Personalized Physiological Forecasting
and Explainable Anomaly Detection

Full Technical Report — Data Pipeline, Feature Engineering,
Models, Anomaly Layer, and System Architecture

Care-Sync Engineering

July 17, 2026

Abstract

Care-Sync is an end-to-end platform that ingests wearable physiological signals (Empatica E4: PPG/BVP, HR, inter-beat intervals, accelerometry, electrodermal activity, and skin temperature), learns what is *normal for a specific individual*, forecasts their heart rate and heart-rate variability (HRV) one, five, and ten minutes ahead, and raises *explainable* health-drift alerts when the measured signal deviates from that personal baseline. This report documents every step of the working system: how raw device files are read, synchronized, filtered, and cleaned; how each feature is extracted (with exact formulas and windowing); how sequence windows and forecast targets are constructed; the forecasting models (TCN, LSTM, GRU, Transformer, XGBoost) and their heteroscedastic β -NLL training; the personalization stack (global pretraining, per-user fine-tuning, self-supervised warm-start, on-line updates, and the digital twin); the multi-channel anomaly detector (adaptive z-scores, Kalman innovation, CUSUM drift, illness composite, signal-quality gating, and hysteresis alerting); and the surrounding platform (FastAPI AI service, Django REST backend, and Next.js frontend).

Contents

1	System Overview	2
1.1	End-to-end data flow	2
2	Data Ingestion	3
3	Synchronization and Resampling	3
4	Signal Cleaning and Filtering	4
4.1	Band-pass (BVP / PPG)	4
4.2	Low-pass (EDA / TEMP)	4
4.3	Gravity removal (ACC)	4
4.4	Range clipping and plausibility	4
4.5	Gap handling and segmentation	4
5	Feature Engineering	4
5.1	HR features (<code>hr_features.py</code>)	5
5.2	IBI / HRV features (<code>ibi_features.py</code>)	5
5.3	Accelerometer features (<code>acc_features.py</code>)	6
5.4	BVP / pulse features (<code>bvp_features.py</code>)	6

5.5	EDA features (<code>eda_features.py</code>)	6
5.6	Temperature features (<code>temp_features.py</code>)	6
5.7	Time and circadian features (<code>time_features.py</code> , <code>circadian.py</code>)	6
5.8	Physiological state classifier (<code>state_classifier.py</code>)	7
5.9	Recovery / fitness features (<code>recovery_features.py</code>)	7
5.10	Subject-relative baselines and long context (<code>subject_baseline.py</code>)	7
5.11	Activity-gating features (<code>activity_gating.py</code>)	7
5.12	Feature selection and forecast targets	8
6	Windowing and Dataset Construction	8
7	Forecasting Models	8
7.1	Temporal Convolutional Network (primary)	8
7.2	Recurrent baselines (LSTM / GRU)	9
7.3	Transformer	9
7.4	XGBoost (tabular baseline)	9
7.5	β -NLL loss and training	9
8	Personalization	9
8.1	Personal Digital Twin (<code>personalization/digital_twin.py</code>)	10
8.2	Kalman baseline tracker (<code>models/kalman.py</code>)	10
9	Anomaly Detection	10
9.1	Uncertainty-normalized residuals	10
9.2	Adaptive (EWMA) baselines with warm-up (<code>adaptive.py</code>)	10
9.3	Multi-channel evidence	11
9.4	Illness composite (<code>explain.py</code>)	11
9.5	Combined score, exercise suppression, illness boost	11
9.6	Signal-quality gating (<code>signal_quality.py</code>)	11
9.7	Hysteresis alerting and threshold calibration (<code>detector.py</code>)	12
9.8	Explainable reasons	12
10	Simulation and Evaluation	12
11	AI Serving Layer	12
11.1	Model resolution (<code>hrv_forecast/model_loader.py</code>)	12
11.2	Feature adapter (<code>hrv_forecast/feature_adapter.py</code>)	12
11.3	Real vs. mock engines and graceful fallback	13
11.4	Agentic analysis workflow (<code>ai/workflow.py</code>)	13
11.5	Standalone PPG analysis (<code>ai/analysis/</code>)	13
12	Backend Platform (Django REST)	13
12.1	Authentication	13
12.2	Data model (<code>login/models.py</code>)	13
12.3	Signal Portal and HRV in the backend	14
12.4	HL7 export and alerts	14
13	Frontend and Device Ingestion	14
13.1	Next.js web app	14
13.2	Device heartbeat	14
14	Configuration Reference	14

1 System Overview

Care-Sync is organized into four cooperating layers plus a device-side helper, each in its own top-level directory of the repository.

Layer	Stack	Responsibility
frontend/	Next.js 16, React 19, TypeScript	Web UI: login, dashboard, files, PPG visualization, the device <i>Signal Portal</i> , trends, alerts, consent, admin. Talks to the backend over <code>/api</code> .
backend/	Django 6, DRF, SimpleJWT	REST API: authentication (email OTP + JWT), file management, signal visualization, HL7 export, device registry, portal aggregation, alert rules, HRV anomaly records, consent.
ai/	FastAPI, LangGraph, PyTorch	Agentic analysis + the HRV forecast/anomaly/digital-twin service. Wraps the research pipeline behind a stable HTTP contract with a deterministic mock fallback.
HRV / personalized_hrv_system/ /	Python, NumPy, pandas, SciPy, PyTorch, XGBoost	The research pipeline: cleaning, feature engineering, models, training, personalization, anomaly scoring, simulation, evaluation.
device-ingestion/ /	stdlib Python	Device heartbeat helper that posts battery/firmware/session status to the backend device API.

1.1 End-to-end data flow

The physiological data path, from sensor to alert, is:

1. **Ingest** raw per-signal files, each carrying its own start time and sample rate in a header (Section 2).
2. **Synchronize** every signal onto a common 1 Hz grid, aggregating fast signals per second (Section 3).
3. **Clean** each modality with signal-appropriate filters, artifact rejection, and gap handling (Section 4).
4. **Engineer features**: rolling HR/HRV statistics, activity, circadian baseline, physiological state, recovery, subject-relative baselines, and long-context summaries (Section 5).
5. **Window** the feature table into input sequences with multi-horizon forecast targets (Section 6).
6. **Forecast** HR (and RMSSD) at +60, +300, +600s with a predicted mean *and* uncertainty (Section 7).
7. **Personalize**: bootstrap from a population model, fine-tune per user, and keep adapting online; summarize the person as a digital twin (Section 8).
8. **Detect anomalies** by comparing actual vs. forecast through a multi-channel, quality-gated, context-aware scorer, and raise explainable alerts (Section 9).
9. **Serve** all of this over HTTP and surface it in the portal (Sections 11–13).

A central design principle is **causality**: every feature and every baseline is computed using only past and current samples (rolling windows, expanding means with a one-step shift, EWMA trackers), so the exact same code that trains offline can run in real time on a streaming device.

2 Data Ingestion

Raw data follows the Empatica E4 export format. Each session is a folder (`backend/Users/<user>/<session>/` in production, or `Dataset3/Raw_data/SXX/` in research) containing one file per signal. Readers live in `src/data/loader.py`.

File	Native rate	Format
HR.csv	~1 Hz	row1 = Unix start ts (UTC), row2 = Hz, then values.
EDA.csv	4 Hz	same 2-row header, electrodermal activity (μS).
TEMP.csv	4 Hz	same header, skin temperature ($^{\circ}\text{C}$).
BVP.csv	64 Hz	same header, raw blood-volume-pulse (PPG) waveform.
ACC.csv	32 Hz	row1 = start ts $\times 3$, row2 = Hz $\times 3$, then x, y, z in units of $1/64g$ (converted to g on read).
IBI.csv	irregular	row0 = <code>start_ts</code> , IBI; then (<code>offset_s</code> , <code>ibi_s</code>) pairs — inter-beat intervals, no fixed Hz.
tags*.csv	—	one Unix timestamp per line (event/button markers).
info.txt	—	metadata, shown verbatim.

Each reader parses the header to build a UTC `DatetimeIndex` from the start timestamp and sample rate:

$$t_i = t_{\text{start}} + \frac{i}{f_s}, \quad i = 0, 1, \dots, N - 1.$$

The ACC reader divides raw counts by 64 to obtain g . The IBI reader places each beat at $t_{\text{start}} + \text{offset}_s$, producing an irregularly-sampled series. All reads are wrapped by `_safe_read`: a missing or malformed file yields an empty Series/DataFrame plus a warning, so the pipeline degrades gracefully to partial data instead of crashing. The loader also auto-resolves both flat (`SXX/HR.csv`) and nested (`<SID>_E4_Data/HR.csv`) layouts.

3 Synchronization and Resampling

`src/data/sync.py::build_synced_frame` aligns all modalities onto a common 1 Hz grid (`cfg.resample.freq = "1s"`). 1 Hz is chosen because HR is natively ~1 Hz, it is fine enough for minute-scale forecasting, and it is cheap enough for on-device streaming. Per-signal handling:

- **HR**: clipped to a plausible range, then `resample("1s").mean()` to absorb tiny timestamp drift.
- **ACC (32 Hz)**: gravity removed (Section 4), then the per-axis magnitude $\|a\| = \sqrt{a_x^2 + a_y^2 + a_z^2}$ is aggregated per second into `mean`, `std`, and `max` (`ACC_mag_mean/std/max`) — capturing both average intensity and burstiness within the second.
- **EDA (4 Hz)**: low-pass smoothed, then per-second mean.
- **TEMP (4 Hz)**: per-second mean (slow, clean signal).
- **BVP (64 Hz)**: band-pass cleaned but *kept at native rate*, so peak detection has full temporal resolution.
- **IBI (irregular)**: implausible beats dropped, kept on its native irregular index for beat-level HRV.

The four grid-aligned columns (HR, EDA, TEMP, ACC aggregates) are concatenated, sorted, and passed through short-gap interpolation. The function returns a dict with `grid`, native-rate `bvp_clean`, irregular `ibi_clean`, and the `tags` index.

4 Signal Cleaning and Filtering

Cleaning is signal-specific (`src/data/cleaning.py`). All frequency-domain filters are zero-phase Butterworth filters applied with `scipy.signal.filtfilt` (forward-backward), which doubles the effective order and, crucially, introduces *no phase delay* — essential when the timing of pulse peaks and HR changes carries physiological meaning.

4.1 Band-pass (BVP / PPG)

BVP is band-pass filtered to $[0.5, 8]$ Hz (order 3) to remove baseline drift below the pulse band and high-frequency noise above it. With Nyquist $f_{\text{nyq}} = f_s/2$, the normalized cutoffs are

$$W_{\text{low}} = \max\left(\frac{0.5}{f_{\text{nyq}}}, 10^{-4}\right), \quad W_{\text{high}} = \min\left(\frac{8}{f_{\text{nyq}}}, 0.999\right),$$

and the biquad coefficients come from `butter(3, [W_low, W_high], "band")`.

4.2 Low-pass (EDA / TEMP)

EDA is low-pass filtered at 1 Hz (order 3) before being split into tonic and phasic components (Section 5). The normalized cutoff is $W_c = \min(f_c/f_{\text{nyq}}, 0.999)$.

4.3 Gravity removal (ACC)

Each accelerometer axis is high-pass filtered at 0.5 Hz to strip the DC gravity component, leaving only *dynamic* acceleration (movement) before the magnitude is computed. This prevents posture (a static tilt) from registering as activity.

4.4 Range clipping and plausibility

- HR is clipped to $[30, 220]$ bpm.
- IBI is filtered to $[0.3, 2.0]$ s (i.e. 30–200 bpm); values outside this are physiologically impossible and are dropped (`clean_ibi`).

4.5 Gap handling and segmentation

`fill_small_gaps` linearly interpolates gaps up to `max_interpolate_gap_s` (5 s). Each interpolated sample is tagged with a `<col>_stale` flag ($= 1$ when the value was filled across a > 1 -sample gap), so the model and the signal-quality layer can down-weight it. Gaps longer than `max_segment_gap_s` (60 s) are treated as true discontinuities: `segment_by_gaps` splits the recording into contiguous segments, and sequence windows are never built across a segment boundary. This means a ~ 1 -hour session broken by charging/removal yields several clean segments rather than one corrupted one.

5 Feature Engineering

`src/features/build_features.py::build_feature_table` orchestrates all feature modules into one per-second table indexed by the 1 Hz grid. Modules are included conditionally on sensor

availability, so a dataset lacking (say) EDA simply omits EDA features. The assembly order matters because later modules consume earlier ones:

1. Base grid + HR features + IBI/HRV features + time features (always).
2. ACC, BVP, TEMP, EDA features (if present).
3. Circadian baseline (needs HR).
4. Physiological state classifier (needs activity + HR + circadian).
5. Recovery features (need state).
6. Subject-relative baselines and long-context summaries.
7. Activity-gating features.
8. Forecast targets (future HR/RMSSD by shifting).

5.1 HR features (`hr_features.py`)

For each window $w \in \{30, 60, 300\}$ s over the HR series $h(t)$:

$$\mu_w^{\text{HR}}(t) = \frac{1}{w} \sum_{\tau=t-w+1}^t h(\tau), \quad \sigma_w^{\text{HR}}(t) = \sqrt{\frac{1}{w-1} \sum_{\tau=t-w+1}^t (h(\tau) - \mu_w^{\text{HR}}(t))^2}.$$

A trailing-window **trend** is the least-squares slope of h over w seconds; with $x = 0, \dots, w-1$ and $\bar{x}$ its mean,

$$\text{slope}_w(t) = \frac{\sum_x (x - \bar{x})(h - \bar{h})}{\sum_x (x - \bar{x})^2} \quad [\text{bpm/s}].$$

Rate-of-change features $\text{HR_roc}_\Delta(t) = h(t) - h(t - \Delta)$ are computed for $\Delta \in \{5, 30, 60\}$ s.

5.2 IBI / HRV features (`ibi_features.py`)

Wrist-PPG HRV is noisy, so beats are cleaned *before* HRV is computed. A **Hampel filter** rejects ectopic/artifact beats: with local median m_i over a 5-beat centered window and median absolute deviation MAD_i , a beat is flagged when

$$|b_i - m_i| > n_\sigma \cdot 1.4826 \cdot \text{MAD}_i, \quad n_\sigma = 3,$$

where 1.4826 makes the MAD a consistent estimator of the standard deviation for Gaussian data. Flagged beats are replaced by the local median, and any successive difference that spans a rejected beat is invalidated. Over each HRV window $w \in \{60, 300\}$ s (requiring $\geq \text{hrv_min_beats}$ valid beats):

$$\begin{aligned} \text{RMSSD}_w &= \sqrt{\frac{1}{M-1} \sum_i (b_{i+1} - b_i)^2} && (\text{short-term parasympathetic HRV}) \\ \text{SDNN}_w &= \text{std}(\{b_i\}) && (\text{overall HRV}) \\ \text{pNN50}_w &= \frac{1}{M} \sum_i \mathbb{1}[|b_{i+1} - b_i| > 50\text{ms}] && (\text{fraction of large beat-to-beat jumps}) \\ \text{HR}_w^{\text{IBI}} &= \frac{60}{\bar{b}} && (\text{HR implied by mean IBI}) \end{aligned}$$

Each beat-indexed metric is forward-filled onto the 1 Hz grid. A quality column `RMSSD_valid_fraction_ws` (the rolling fraction of non-rejected beats) is emitted for the signal-quality layer.

5.3 Accelerometer features (`acc_features.py`)

From `ACC_mag_mean` (gravity-removed magnitude in g), rolling intensity and its std are computed over $w \in \{10, 60\}$ s. The primary intensity is bucketed into an ordinal **activity level** using the thresholds $\{0.05, 0.2, 0.5\} g$:

$$\text{bucket} \in \{\text{REST}, \text{LIGHT}, \text{MODERATE}, \text{VIGOROUS}\},$$

plus a binary `movement_flag` = $\mathbb{K}[\text{intensity} > 0.05 g]$.

5.4 BVP / pulse features (`bvp_features.py`)

On the native-rate cleaned BVP, systolic peaks are found with `scipy.signal.find_peaks`, enforcing a minimum inter-peak distance of $\lfloor f_s \cdot 60/220 \rfloor$ samples (a 220 bpm ceiling). Peak *prominences* give a pulse-amplitude series. Per second, the pulse count is resampled and a rolling 60 s sum yields `BVP_pulse_rate_1min`; rolling 30 s mean/std of amplitude give `BVP_amplitude_mean/std_30s`.

5.5 EDA features (`eda_features.py`)

The tonic (slow) level is a 60 s rolling mean; the phasic (fast, SCR) component is the residual:

$$\text{EDA}_{\text{tonic}}(t) = \frac{1}{60} \sum_{\tau=t-59}^t \text{EDA}(\tau), \quad \text{EDA}_{\text{phasic}}(t) = \text{EDA}(t) - \text{EDA}_{\text{tonic}}(t),$$

with a rolling mean of $|\text{phasic}|$ as a 60 s arousal proxy.

5.6 Temperature features (`temp_features.py`)

A 300 s rolling mean and slope capture drift; deviation from a personal baseline uses an *expanding* mean (a proxy for the calibration-period baseline):

$$\text{TEMP_baseline_deviation}(t) = \text{TEMP}(t) - \frac{1}{t} \sum_{\tau \leq t} \text{TEMP}(\tau).$$

5.7 Time and circadian features (`time_features.py`, `circadian.py`)

Time-of-day and day-of-week are encoded cyclically so midnight is adjacent to 23:59:

$$\left(\sin \frac{2\pi H}{24}, \cos \frac{2\pi H}{24} \right), \quad \left(\sin \frac{2\pi D}{7}, \cos \frac{2\pi D}{7} \right),$$

where H is fractional hour and D day-of-week. The **causal circadian baseline** answers “what is normal HR for this person at this hour?” For each hour bucket $h \in \{0, \dots, 23\}$ it maintains an expanding mean/std of HR, *shifted by one step* so the current sample never sees itself:

$$z^{\text{circ}}(t) = \frac{\text{HR}(t) - \mu_{h(t)}^{\text{circ}}(t^-)}{\max(\sigma_{h(t)}^{\text{circ}}(t^-), 10^{-6})}.$$

On a short single session these are degenerate (one hour bucket), but the same code scales directly to weeks of continuous data where each bucket accumulates many prior days.

5.8 Physiological state classifier (`state_classifier.py`)

Each 1 Hz sample is labeled with one of *sleep*, *rest*, *focused_work*, *walking*, *exercise*, *recovery*, *unknown*. This gives the anomaly layer context: an HR spike during *exercise* is expected; the same spike during *sleep* is not. The decision cascade (all causal):

1. activity $\geq$ MODERATE $\Rightarrow$ **exercise**.
2. recent exercise (600 s lookback) and HR still $>$ circadian mean +3 $\Rightarrow$ **recovery**.
3. activity = LIGHT $\Rightarrow$ **walking**.
4. activity = REST, night hours (23:00–05:59), HR $\leq$ circadian mean $\Rightarrow$ **sleep**.
5. activity = REST, daytime, elevated EDA arousal $\Rightarrow$ **focused_work**.
6. activity = REST otherwise $\Rightarrow$ **rest**.

Missing inputs degrade gracefully to *unknown*.

5.9 Recovery / fitness features (`recovery_features.py`)

During and after an exercise bout the module tracks the bout’s peak HR and how fast HR falls from it. Heart-rate recovery is a well-known fitness marker:

$$\text{HRR}(t) = \max(0, \text{HR}_{\text{peak}} - \text{HR}(t)), \quad \text{HRR rate}(t) = \frac{\text{HRR}(t)}{\Delta t_{\text{since peak}}/60} \text{ [bpm/min]}.$$

A faster drop indicates better cardiovascular fitness; persistently slow recovery over weeks flags declining fitness or incomplete recovery. All outputs default to 0 outside a bout, so no NaNs are introduced.

5.10 Subject-relative baselines and long context (`subject_baseline.py`)

Absolute HR/TEMP/RMSSD vary enormously between people, so deviation from a person’s *own* recent baseline is more informative than the raw value. For each signal s and baseline window $W = 3600$ s (causal):

$$s_{\text{minus_baseline}}(t) = s(t) - \mu_W^s(t), \quad s_{\text{z_rolling}}(t) = \frac{s(t) - \mu_W^s(t)}{\sigma_W^s(t)}.$$

Multi-scale long-context summaries of HR and RMSSD over $W \in \{900, 1800, 3600\}$ s (15/30/60 min) provide `ctx_mean`, `ctx_std`, and an endpoint slope $(s(t) - s(t - W))/W$, giving the model both the immediate trajectory and the slow baseline trend without needing a 60-minute raw input window. All deviation columns default to 0 (never NaN) so windowing never drops rows.

5.11 Activity-gating features (`activity_gating.py`)

“High HR” means different things at different motion levels: HR-high + ACC-high is likely exercise; HR-high + ACC-low is more suspicious (stress, illness, arousal). For each activity bucket, a strictly-causal running mean/variance of HR is maintained with **Welford’s algorithm**:

$$\mu_k = \mu_{k-1} + \frac{x_k - \mu_{k-1}}{k}, \quad M_{2,k} = M_{2,k-1} + (x_k - \mu_{k-1})(x_k - \mu_k), \quad \sigma_k^2 = \frac{M_{2,k}}{k}.$$

This yields `hr_expected_for_activity`, the residual, a clipped `hr_z_for_activity` $\in [-8, 8]$, and an explicit `hr_high_while_still` flag (low motion but elevated HR-for-activity). NaN HR (sensor gaps) never enters the running stats, so a gap cannot poison the baseline.

5.12 Feature selection and forecast targets

`numeric_feature_columns` selects the model inputs: it keeps every numeric column and drops (i) target columns (any `*_target_*`) and (ii) the categorical name columns (`activity_bucket_name`, `physio_state_name`). Forecast **targets** are built by shifting HR (and, when `predict_rmssd` is on, RMSSD) backward by each horizon:

$$\text{HR_target}_h(t) = \text{HR}(t + h), \quad h \in \{60, 300, 600\} \text{ s.}$$

Optional targets (config flags) add change-from-now deltas ($\text{HR}(t + h) - \text{HR}(t)$) and other vitals (TEMP/EDA) for multi-task learning.

6 Windowing and Dataset Construction

`src/models/datasets.py::make_windows` slides a length- L window (`input_seq_len_s = 120` s) over the feature table with a stride (`stride_s = 5` s), producing

$$X \in \mathbb{R}^{N \times L \times F}, \quad y \in \mathbb{R}^{N \times H}, \quad \text{end_idx} \in \mathbb{Z}^N,$$

where F is the number of features and H the number of horizons. A window is *dropped* if any feature or target within it is NaN — this is exactly why so many features are engineered to default to 0 rather than NaN. The short 120 s context is deliberate: it keeps fragmented ~ 1 -hour sessions productive while the long-context features (above) carry the slow information a longer raw window would otherwise be needed for.

Chronological split. Data is split forward in time (never shuffled across time) into train/-val/test by `val_fraction` and `test_fraction` (0.15 each), so evaluation always predicts the future from the past.

Scaling. A `StandardScaler` is fit on the flattened training windows ($N \cdot L, F$) and applied to all splits. Targets are *also* standardized (`fit_target_scaler`), because HR (bpm) and RMSSD (ms) live on very different scales and mixing them raw destabilizes the Gaussian-NLL objective. Predictions are inverse-transformed back to physical units for reporting; the predicted std is rescaled by the per-target scale, $\sigma_{\text{raw}} = \sigma_{\text{scaled}} \cdot \text{scale}$.

7 Forecasting Models

All sequence models share a **heteroscedastic head**: from the representation at the most recent timestep they emit both a mean and a positive standard deviation per horizon,

$$\hat{\mu}_h, \hat{\sigma}_h = \text{softplus}(\cdot) + 10^{-3},$$

so every forecast comes with its own uncertainty — the quantity the anomaly layer normalizes by.

7.1 Temporal Convolutional Network (primary)

The recommended model is a causal dilated TCN (`src/models/tcn.py`). Each `CausalConvBlock` is a 1-D convolution with left-padding $(k - 1) \cdot d$ that is trimmed on the right (*causal trim*), a ReLU, dropout, and a residual connection:

$$\text{out} = \text{Dropout}(\text{ReLU}(\text{CausalConv}_{k,d}(x))) + \text{Res}(x).$$

Stacking $\ell = 0, \dots, L-1$ levels with exponential dilation $d = 2^\ell$ gives an exponentially growing receptive field

$$\text{RF} = 1 + \sum_{\ell=0}^{L-1} 2(k-1)2^\ell.$$

With the tuned config ($k = 3$, levels = 4) the RF is ≈ 61 s, matched to the 120 s window — an earlier $k = 5$, 6-level version had RF ≈ 505 s $\gg$ window and overfit. Hidden width is 32 (down from 64 to avoid overfitting on a few hundred windows/subject).

7.2 Recurrent baselines (LSTM / GRU)

`src/models/lstm_gru.py` provides an LSTM and a GRU sharing the same mean+std head, reading the sequence and predicting from the last hidden state.

7.3 Transformer

`src/models/transformer.py` projects features to d_{model} , adds sinusoidal positional encoding, and applies a causal-masked `TransformerEncoder` so predictions depend only on past/current steps.

7.4 XGBoost (tabular baseline)

`src/models/xgb_model.py` trains one gradient-boosted regressor per target on a single feature vector per timestep (the engineered rolling features already encode recent history), so it needs no sequence window. Uncertainty is approximated by the constant per-target residual std on the validation set.

7.5 β -NLL loss and training

Models are trained (`src/models/train.py`) by minimizing the β -NLL objective (Seitzer et al., 2022):

$$\mathcal{L} = \frac{1}{NH} \sum_{i,h} \underbrace{\left(\frac{1}{2} \log(2\pi\hat{\sigma}_{ih}^2) + \frac{(y_{ih} - \hat{\mu}_{ih})^2}{2\hat{\sigma}_{ih}^2} \right)}_{\text{Gaussian NLL}} \cdot (\hat{\sigma}_{ih}^2)_{\text{stop-grad}}^\beta, \quad \beta = 0.5.$$

Plain Gaussian NLL can be minimized by *inflating* the variance (predicting the mean with huge σ), because the $1/\sigma^2$ term suppresses the error — the “variance collapse” that makes forecasts flat. Multiplying by $\text{stop-grad}(\sigma^2)^\beta$ restores the mean’s gradient magnitude; $\beta=0$ recovers standard NLL, $\beta=1$ is MSE on the mean. Optimization uses Adam; **model selection is by validation MSE of the mean**, not NLL, so a variance-collapsed model can never appear “best.” Forecast quality is reported per horizon as MAE, RMSE, and MAPE.

8 Personalization

The system learns *per person* through a staged transfer path (`src/pipeline/`).

Stage	What it does
Global pretraining (<code>pretrain_pipeline.py</code>)	Pools windows from many subjects, intersects their feature columns, trains one population model. Serves every new user until they have enough personal data.
SSL warm-start (<code>ssl_pretrain_pipeline.py</code>)	Self-supervised masked reconstruction: randomly mask 25% of feature values and train a TCN backbone to reconstruct them (no labels). The learned backbone warm-starts a forecaster, helping subjects with little data.
Per-user fine-tune (<code>finetune_pipeline.py</code>)	Continues training the pretrained model on one subject’s windows (optionally freezing the backbone), in the pretrained target-scaler space. Saved as a normal loadable checkpoint.

Stage	What it does
Online (<code>online_update_pipeline.py</code>)	update Periodically (e.g. nightly) continues training on the <i>most recent</i> windows only, at a small LR for a few epochs, and <code>partial_fits</code> the feature scaler — so the model tracks slow shifts (e.g. improving fitness lowering resting HR over months).
Multi-dataset (<code>multi_dataset_pipeline.py</code>)	harness Leave-one-dataset-out and repeated-split evaluation across pooled datasets.

8.1 Personal Digital Twin (`personalization/digital_twin.py`)

The twin is a compact JSON profile of a person’s own physiology: resting, sleep, walking, and running HR (means per physiological state), average RMSSD, and a per-hour-of-day circadian HR table. The **health-drift score** compares the live HR to this personal expectation:

$$\text{DigitalTwinScore}(t) = \frac{\text{HR}(t) - \text{ExpectedHR}(t)}{\text{ExpectedStd}(t)},$$

where $\text{ExpectedHR}(t)$ is read from the circadian table for the current hour, falling back to the state-specific baseline (sleep/rest/walk/run) when no circadian estimate exists yet. This personal baseline — not a population norm — is the reference for drift.

8.2 Kalman baseline tracker (`models/kalman.py`)

A 1-D constant-velocity Kalman filter tracks a slowly-varying scalar (resting HR, or the forecast residual) with state [level, trend]:

$$x_k = Fx_{k-1} + w, \quad w \sim \mathcal{N}(0, Q); \quad z_k = Hx_k + v, \quad v \sim \mathcal{N}(0, R).$$

The normalized innovation $\tilde{y}_k = (z_k - H\hat{x}_k^-)/\sqrt{S_k}$ is used by the anomaly layer to catch gradual drift.

9 Anomaly Detection

The anomaly layer (`src/anomaly/`) is deliberately *separate* from the forecaster: it consumes the model’s per-timestep outputs $(y_t, \hat{\mu}_t, \hat{\sigma}_t)$ and turns them into an interpretable score and alerts, saving every intermediate quantity for debugging.

9.1 Uncertainty-normalized residuals

The core signal is the residual normalized by the model’s own predicted uncertainty (`detector.py`):

$$z_h(t) = \frac{|y_h(t) - \hat{\mu}_h(t)|}{\hat{\sigma}_h(t) + \varepsilon}, \quad \text{clipped to } [0, 8].$$

Clipping prevents a single artifact from exploding the score. Per-target z -scores are blended with separate HR/RMSSD weights and a mild shorter-horizon emphasis.

9.2 Adaptive (EWMA) baselines with warm-up (`adaptive.py`)

“Normal” is contextual and drifts, so residual statistics are tracked with an exponentially-weighted mean/variance, optionally *per activity bucket*:

$$\mu \leftarrow \mu + \lambda(x - \mu), \quad \sigma^2 \leftarrow (1 - \lambda)(\sigma^2 + \lambda(x - \mu)^2), \quad \lambda = 0.02.$$

Variance is initialized large ($\sigma \approx 10$ bpm) and z -scores are suppressed to 0 for the first 50 observations per key, killing cold-start false alarms.

9.3 Multi-channel evidence

The scorer (`scoring.py`) combines complementary detectors, each catching a different failure mode:

- **Forecast z -scores** — point deviations from the personal forecast, per horizon.
- **Prediction-interval score** $|y - \hat{\mu}|/\hat{\sigma}$ and $\pm 1.96\hat{\sigma}$ out-of-interval flags.
- **RMSSD drop** — a one-sided z -score and fractional drop of HRV below its EWMA baseline (HRV collapse).
- **Kalman innovation** — normalized innovation of the 1-min residual stream (slow drift).
- **CUSUM drift** — a two-sided decaying CUSUM on residuals that accumulates sustained directional bias per-step z -scores normalize away:

$$s_i^+ = \max(0, \text{decay} \cdot s_{i-1}^+ + z_i - k), \quad s_i^- = \max(0, \text{decay} \cdot s_{i-1}^- - z_i - k),$$

with slack $k = 0.7$ and $\text{decay} = 0.998$ (≈ 6 -min half-life), so exercise bursts do not bleed into later rest.

- **Circadian z and digital-twin z** — deviation from the person’s time-of-day and state baselines.

9.4 Illness composite (`explain.py`)

Three signals classically co-move during infection/fever; individually each may be noise, together they are strong evidence:

$$\text{illness} = \frac{1}{3} \left(\frac{[z^{\text{HR}}]_+}{3} + \frac{[\text{RMSSD}_{\% \text{drop}}]_+}{0.35} + \frac{[\Delta \text{TEMP}]_+}{1.0} \right),$$

scaled so a “typical fever” (HR $z \approx 3$, RMSSD drop $\approx 35\%$, TEMP $+1^\circ\text{C}$) scores ≈ 1.0 .

9.5 Combined score, exercise suppression, illness boost

$$\text{score}(t) = \max_{\text{channels}} \left(w_{\text{sup}}(t) \cdot \text{channel} \right) + 1.5 \cdot [\text{illness}(t)]_+.$$

During *exercise/recovery* the HR-based channels (forecast, circadian, digital-twin, Kalman, CUSUM) are down-weighted by 80% ($w_{\text{sup}} = 0.2$) because elevated HR is expected; RMSSD and illness terms are *not* suppressed. The base score is a max over channels so one strong signal fires immediately, while the additive illness term lets several individually-mild channels jointly cross the threshold.

9.6 Signal-quality gating (`signal_quality.py`)

Real wrist data is full of motion/contact artifacts; without a quality gate, sensor noise is mistaken for a health event. A per-second Signal-Quality Index $\in [0, 1]$ is computed per modality (implausible jumps, out-of-range values, stale/interpolated flags, HRV valid-fraction, ACC saturation, BVP flatline). The overall SQI is the *weakest* modality (a single bad sensor spoils the window), and the effective anomaly score is

$$\text{score}_{\text{eff}}(t) = \text{score}(t) \cdot \text{SQI}_{\text{overall}}(t),$$

so unreliable windows are labeled “sensor issue,” not “anomaly.”

9.7 Hysteresis alerting and threshold calibration (detector.py)

Alerts use a start/stop hysteresis state machine to avoid chatter:

- An alert **starts** only after $\text{score}_{\text{eff}} \geq \theta_{\text{start}}$ for `min_duration_s` continuous seconds (the build-up is backfilled).
- It **stays** until the score drops below $\theta_{\text{stop}} = 0.6 \theta_{\text{start}}$.
- After it ends, a `cooldown_s` quiet period applies, bypassed only for genuine emergencies ($\geq 1.5 \theta_{\text{start}}$).

The start threshold is **calibrated** on the distribution of *normal* (non-anomalous) validation scores at a chosen percentile (99th, floored at 3.0), targeting a false-alert budget rather than a hand-picked number. `alert_stats` reports the normal alert rate, alerts-per-hour, and mean alert duration; `severity` maps the continuous score to *normal* / *watch* / *alert* / *urgent*. In the served AI path, severity is bucketed as $\text{normal} < 1.5 \leq \text{watch} < 3.0 \leq \text{alert}$.

9.8 Explainable reasons

Every alert carries a human-readable reason built from the same numbers, e.g. “+HR 22bpm above expected, RMSSD dropped 35%, Temp elevated +1.2°C, Activity: rest”, so a clinician sees *why*, not just a score.

10 Simulation and Evaluation

`src/simulation/` generates synthetic 1 Hz traces (circadian HR baseline + activity-driven response) with injected abnormal windows labeled by an `anomaly_label` column, and can concatenate scenarios onto a real subject’s grid for “inject-into-real-data” testing. Evaluation scripts (`scripts/run_*.py`) cover preprocessing, training, inference, ablation, error analysis, online update, and simulation scoring. Forecast metrics are per-horizon MAE/RMSE/MAPE; calibration (`evaluation/calibration.py`) checks that predicted uncertainty matches observed error; the anomaly harness reports detection rate and false-alarms-per-hour.

11 AI Serving Layer

The `ai/` FastAPI service exposes the research pipeline over HTTP with a stable contract and a never-fail fallback.

11.1 Model resolution (hrv_forecast/model_loader.py)

`HRV_MODEL_DIR` is treated as a *base* directory. Per request, a checkpoint is resolved in priority order: the subject’s personal directory → `global/` → the base directory as a flat checkpoint. Each resolved directory is loaded once and cached, so many subjects sharing the global model pay the load cost only once. Any load error is logged and the service falls back to the mock engine rather than crashing.

11.2 Feature adapter (hrv_forecast/feature_adapter.py)

The adapter turns an API list of `HRVSamples` into the exact feature table the research pipeline expects — without raw E4 files — by building the “synced” shape (grid, cleaned IBI) directly and calling the real `build_feature_table`. The HR grid is built by resampling first and then reindexing onto the 1 Hz bins (a fix ensuring sub-second-timestamp inputs no longer produce an all-NaN HR column).

11.3 Real vs. mock engines and graceful fallback

`real_engine.py` runs the true pipeline: anomaly scoring and the digital twin need only feature engineering (no trained net) and run for real today; forecasting needs a trained checkpoint and raises if none is loaded. `service.py` wraps each call so that if the real path raises for any reason (too few samples, a bad checkpoint, an unexpected shape), it logs and returns the deterministic mock engine’s response — callers always learn which they got via `model_status`. Endpoints (`routes/hrv.py`): GET `/hrv/status`, POST `/hrv/forecast`, POST `/hrv/anomaly`, POST `/hrv/digital-twin`; plus `/health` and the agentic `/chat`.

11.4 Agentic analysis workflow (`ai/workflow.py`)

A LangGraph state machine chains five nodes:

planner → signal → medical → trend → report → END.

Planner classifies intent (stress/sleep/hrv/trend/general) by keyword; *signal* runs the PPG analysis pipeline; *medical* composes only data-grounded reasoning (never invented numbers); *trend* computes a real first-half-vs-second-half comparison; *report* calls the LLM (`langchain-xai`, Grok) with a strict “use only measured data” system prompt and falls back to a deterministic template if the LLM is unavailable.

11.5 Standalone PPG analysis (`ai/analysis/`)

`HealthPipeline` preprocesses PPG (band-pass 0.5–8 Hz, normalize, Savitzky–Golay smooth), detects peaks (`find_peaks`, min distance 0.45 s), forms RR intervals, and computes HRV time-domain metrics (RMSSD, SDNN, mean RR, mean HR, pNN50) and a heuristic stress score $\text{clip}([60 - \text{RMSSD}]_+ + [\text{HR} - 70]_+, 0, 100)$ mapped to Low/Moderate/High. Numeric results are cast to native Python floats so NumPy scalar reprs never leak into reports.

12 Backend Platform (Django REST)

The `backend/` app (`login/`) is API-only (DRF, JWT via SimpleJWT), grouped under `/api/` in `mHealth/urls.py`.

12.1 Authentication

Email-OTP signup and login with JWT access/refresh tokens: `/api/auth/signup`, `send-otp`, `verify-otp`, `login`, `login/verify`, `logout`, `change-password`, `me`, `token/refresh`. `DJANGO_SECRET_KEY` is environment-only and startup fails loudly in production if unset; `DEBUG` defaults to *False* (fail-closed). Settings now load `backend/.env` explicitly via `load_dotenv` so hand-launched servers honor it.

12.2 Data model (`login/models.py`)

Model	Purpose
<code>UserProfile</code>	Extended per-user profile.
<code>EmailOTP</code>	One-time passcodes for signup/login.
<code>Device</code>	Registered ingestion device (key, battery, firmware, session).
<code>GoogleSheet</code> , <code>ExcelFile</code>	External/tabular data sources.

Model	Purpose
UploadedDocument	Category metadata (lab/prescription/imaging/...) for files on disk.
HRVAnomalyAlert	A fired anomaly/digital-twin deviation from /hrv/anomaly (score, reasons, severity, emailed flag).
AlertRule, AlertFired	Per-user threshold rules and their firings.
SessionAnnotation	Timestamped notes on a session.
ContactMessage, ParticipantConsent	Contact form, health consent.

12.3 Signal Portal and HRV in the backend

The portal (`/api/device/sessions/*`) aggregates E4 session folders: per-minute/hour/day stats for HR/EDA/TEMP, ACC movement magnitude, a BVP waveform sample, and HRV from IBI. Backend HRV mirrors the research formulas: $RMSSD = \sqrt{(\Delta b)^2}$ and $SDNN = \text{std}(b)$ (`views/device.py::_hrv_stats`). `session_trends` and `compare_sessions` power the trends/compare pages.

12.4 HL7 export and alerts

`/api/hl7/*` converts session CSVs into HL7 messages (text and PDF via ReportLab). `/api/alerts/*` manages threshold rules, evaluates them when a session loads, records firings, and ingests HRV anomalies from the AI service (`report_hrv_anomaly`), emailing the user on *alert* severity.

13 Frontend and Device Ingestion

13.1 Next.js web app

The App-Router frontend (`frontend/src/app/`) provides pages for *login*, *dashboard*, *files*, *PPG visualization*, *portal* (+*session/trends/compare*), *alerts*, *devices*, *consent*, and *admin* (overview, patients). `src/lib/api.ts` is an Axios client with a 15 s timeout that attaches the JWT and transparently refreshes on 401. `HRVInsights.tsx` calls the AI service directly (with an 8 s timeout) to render forecasts, the digital twin, and anomaly badges alongside the recharts time series. Styling uses the emerald “CareVibe” theme.

13.2 Device heartbeat

`device-ingestion/heartbeat.py` posts `battery/firmware/session` to `/api/devices/heartbeat/` with an `X-Device-Key` header, so the portal can show a device as online with live status. The key is issued once in the portal’s “Register device” flow.

14 Configuration Reference

Key parameters (`configs/config.yaml`); the AI feature adapter mirrors the relevant subset as a literal dict so `ai/` needs no YAML dependency.

Parameter	Value	Meaning
<code>resample.freq</code>	<code>1s</code>	common grid frequency
<code>hr_min/max_bpm</code>	<code>30 / 220</code>	HR clip range

Parameter	Value	Meaning
ibi_min/max_s	0.3 / 2.0	plausible IBI range
bvp_bandpass_hz	[0.5, 8.0]	BVP band-pass
eda_lowpass_hz	1.0	EDA low-pass
max_interpolate_gap_s	5	max gap to interpolate
max_segment_gap_s	60	gap that splits a recording
hr_windows_s	30/60/300	HR rolling windows
hrv_windows_s	60/300	HRV rolling windows
hrv_min_beats	3	min valid beats per HRV window
hrv_hampel_window/sigma	5 / 3.0	ectopic-beat rejection
baseline_windows_s	3600	personal baseline window
longctx_windows_s	900/1800/3600	long-context summaries
input_seq_len_s	120	model input window
horizons_s	60/300/600	forecast horizons
stride_s	5	window stride
hidden_channels	32	model width
tcn_levels / kernel_size	4 / 3	TCN depth/kernel (RF $\approx$ 61 s)
dropout	0.2	regularization
epochs / lr / batch	40 / 1e-3 / 64	training
nll_beta	0.5	β -NLL weight
val/test_fraction	0.15 / 0.15	chronological split
ewma_lambda	0.02	residual EWMA rate
z_thresh	3.0	base alert threshold
start_percentile	99.0	calibration percentile
stop_fraction	0.6	hysteresis stop ratio
min_duration_s / cooldown_s	20 / 60	alert persistence/quiet

15 Reliability and Hardening Notes

Several robustness properties are worth calling out because they shape the working code throughout:

- **Never-NaN features.** Deviation/recovery/context features default to 0 rather than NaN so `make_windows` does not silently discard rows.
- **Strict causality.** Circadian, activity-gating, and EWMA baselines use expanding means with a one-step shift or online updates, so no feature sees the future.
- **Fail-open service.** The AI HRV endpoints always answer: real pipeline first, deterministic mock on any exception, with the mode exposed via `model_status`.
- **Fail-closed security.** `DEBUG` defaults to False and the secret key must be supplied in production or startup aborts.
- **Safe checkpoint loading.** `torch.load(..., weights_only=True)` is used when loading model/SSL state to avoid arbitrary-code-execution from pickled checkpoints.

This report documents the Care-Sync/M-Health codebase as implemented: the data path (ingest $\rightarrow$ sync $\rightarrow$ clean $\rightarrow$ features $\rightarrow$ windows), the forecasting models and their β -NLL training, the personalization stack and digital twin, the multi-channel explainable anomaly detector, and the FastAPI/Django/Next.js platform that serves it.